Blatt 02: Git-Quest & Calculator (Git Basics, Lambda-Ausdrücke, Gradle)

Carsten Gips (HSBI)

Unterlagen

- PDF-Version

---

1 Git

1.1 Git Status erklären

Betrachten Sie die folgende Ausgabe von `git status` in einer lokalen Workingcopy (*Arbeitskopie*):

```
pm-lecture % git status
On branch b03

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   CONTRIBUTING.md
        modified:   homework/b03.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        foo.java

no changes added to commit (use "git add" and/or "git commit -a")
```

Erklären Sie die Ausgabe.

Geben Sie eine Befehlssequenz an, mit der Sie nur die Änderungen in `foo.java` committen können.

1.2 Git-Spiel

Klonen Sie die Vorgaben "Git-Quest". Sie finden die Geschichte des Helden Markus im Dungeon.[1]

1. Öffnen Sie eine Konsole und beantworten Sie mit Hilfe der Befehle `git checkout`, `git log` und `git show` sowie `git diff` folgende Fragen:

   - Was passierte an `tag 01`?
   - Wann hat der Held zum ersten Mal 4 `experience` Punkte?
   - Wann hat der Held zum ersten Mal 10 `hunger` Punkte?
   - Wie viele Heiltränke hat der Held insgesamt in seinem Rucksack gehabt?
   - Was hat der Held im Shop gekauft? Und wie viel Gold hat er dafür bezahlt?
   - Was passierte zwischen `tag 03` und `tag 04`, d.h. was änderte sich zwischen diesen Commits?
   - Hat der Held etwas gegessen? Falls ja, was und wann?

[1] Für alle, die schon mit Branches umgehen können: Betrachten Sie auf diesem Blatt bitte nur den Branch `master`.

2. Beim letzten Commit (`tag 04.5`) ist etwas schief gelaufen, es wurden versehentlich zu wenig `experience` Punkte eingestellt. Ändern Sie diesen letzten Commit und passen Sie die `experience` Punkte auf 42 an.
3. Schreiben Sie die Geschichte in der Datei `questlog.md` fort und erzeugen Sie einen neuen Commit für `tag 04.6`. Ändern Sie bitte hierzu nur die eine Datei `questlog.md`.
4. Schreiben Sie die Geschichte noch weiter fort (`tag 04.7`), aber ändern Sie diesmal mehrere Dateien, die an diesem Tag (neuer Commit) gemeinsam eingecheckt werden sollen.
5. Fälschlicherweise wurden die Statuspunkte und die Ausrüstung bisher gemeinsam in der Datei `stats.md` geführt. Korrigieren Sie das und verschieben Sie die Ausrüstungsgegenstände aus der Datei `stats.md` in eine neue Datei `gear.md`. Checken Sie Ihre Änderungen als `tag 04.8` (neuer Commit) gemeinsam ein. (*Hinweis*: Es reicht, wenn diese Änderung als letzter Commit auf der Spitze des `master`-Branches existiert. Sie brauchen/sollen die Trennung von Statuspunkten und Ausrüstung **nicht rückwirkend** in die Historie einbauen!)

Demonstrieren Sie Ihr Vorgehen im Praktikum jeweils live.

1.3 Commit-Meldungen

Gute Commit-Meldungen schreiben erfordert Übung. Schauen Sie sich die beiden Commits Dungeon-CampusMinden/Dungeon/commit/46530b6 und Dungeon-CampusMinden/Dungeon/commit/3e37472 an.

Diskutieren Sie jeweils, was Ihnen an den Commits auffällt: Was gefällt Ihnen, was stört Sie? Schlagen Sie Verbesserungen vor.

2 Gradle

Folgen Sie der Anleitung auf gradle.org und installieren Sie Gradle auf Ihrem Rechner. Legen Sie in der Konsole ein neues Gradle-Projekt für eine Java-Applikation an (ohne IDE!). Das Build-Script soll in Groovy erzeugt und als Test-API soll JUnit4 verwendet werden.

Wie finden Sie auf der Konsole heraus, welche Tasks es gibt? Erklären Sie das Projektlayout, d.h. wo kommen beispielsweise die Java-Dateien hin?

Erklären Sie, in welche Abschnitte das generierte Buildskript unterteilt ist und welche Aufgaben diese Abschnitte jeweils erfüllen. Gehen Sie dabei im *Detail* auf das Plugin `application` und die dort bereitgestellten Tasks und deren Abhängigkeiten untereinander ein.

Öffnen Sie das Projekt in Ihrer IDE. Wie können Sie hier die verschiedenen Tasks ansteuern?

Machen Sie sich Notizen, welche Sie im Praktikum nutzen dürfen, um dort das Buildskript zu erklären.

3 Calculator: Anonyme Klassen und Lambda-Ausdrücke

Klonen Sie die Vorgaben "Calculator" und laden Sie das Projekt als Gradle-Projekt in Ihre IDE.

In den Vorgaben wird das Package `calculator` verwendet. Recherchieren Sie und erklären Sie im Praktikum, was Packages in Java sind und wie man damit die Code-Basis strukturieren kann. Gehen Sie dabei auch auf Sichtbarkeiten ein.

Im Package `calculator` finden Sie einige Interfaces und Klassen, mit denen man einen einfachen Taschenrechner modellieren kann: Dieser kann einfache mathematische Operationen auf zwei Integern ausführen.

In der Klasse `calculator.Calculator` finden Sie vier mit `TODO` markierte Stellen in der Methode `setupOperationSelector`:

1. Erstellen Sie eine neue **Java-Klasse** `Sub`, die das Interface `Operation` implementiert und eine Subtraktion bereitstellt. Erweitern Sie den `Calculator` und binden Sie eine **Instanz dieser Klasse** ein. Nutzen Sie hier keine anonymen Klassen oder Lambda-Ausdrücke.
2. Erstellen Sie eine weitere Operation "Mul" (Multiplikation von zwei Integern). Nutzen Sie dazu eine passende **anonyme Klasse**.
3. Erstellen Sie eine weitere Operation "Div" (Integerdivision). Erstellen Sie einen passenden **Lambda-Ausdruck**.
4. Für die `JComboBox` `operationSelector` wird ein `ActionListener` mit Hilfe einer *anonymen Klasse* definiert. Konvertieren Sie dies in einen entsprechenden **Lambda-Ausdruck**.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 8aa3159 (tooling: do not generate beamer slides for readme's and homework, 2025-05-11)